



Universidad Nacional de La Plata



Facultad de Informática

Ejemplo de aplicación

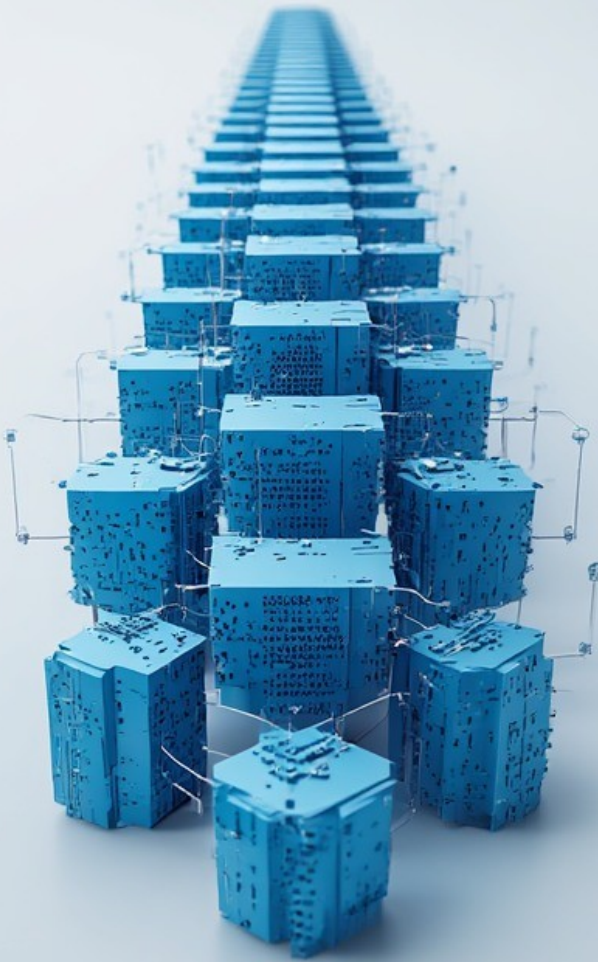
Reducción en CUDA (Optimizado)



1

Reducción

Mejoras por el uso de memoria compartida





1

Reducción

Mejoras por el uso de memoria compartida

Suma por reducción

Mejoras por el uso de memoria compartida

4

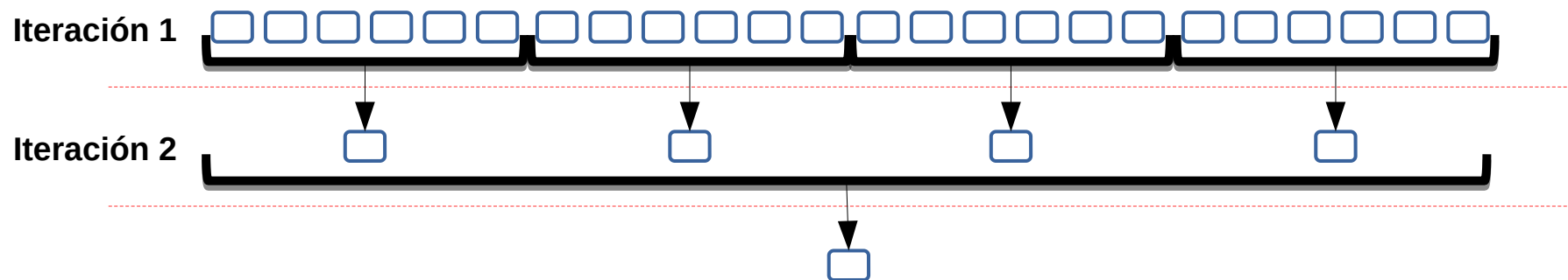
- Podemos mejorar el tiempo de ejecución utilizando una memoria más rápida como la memoria compartida.
- La estrategia es la siguiente:
 - Cada bloque trae una porción de los datos, de forma coalescente, desde memoria global a la memoria compartida.
 - Cada bloque realiza la reducción en memoria compartida.
 - Cada bloque almacena el resultado que reside en memoria compartida en la memoria global.
- Cada bloque dejará un valor. Luego, los resultados de todos los bloques deben reducirse nuevamente.

Suma por reducción

Mejoras por el uso de memoria compartida

6

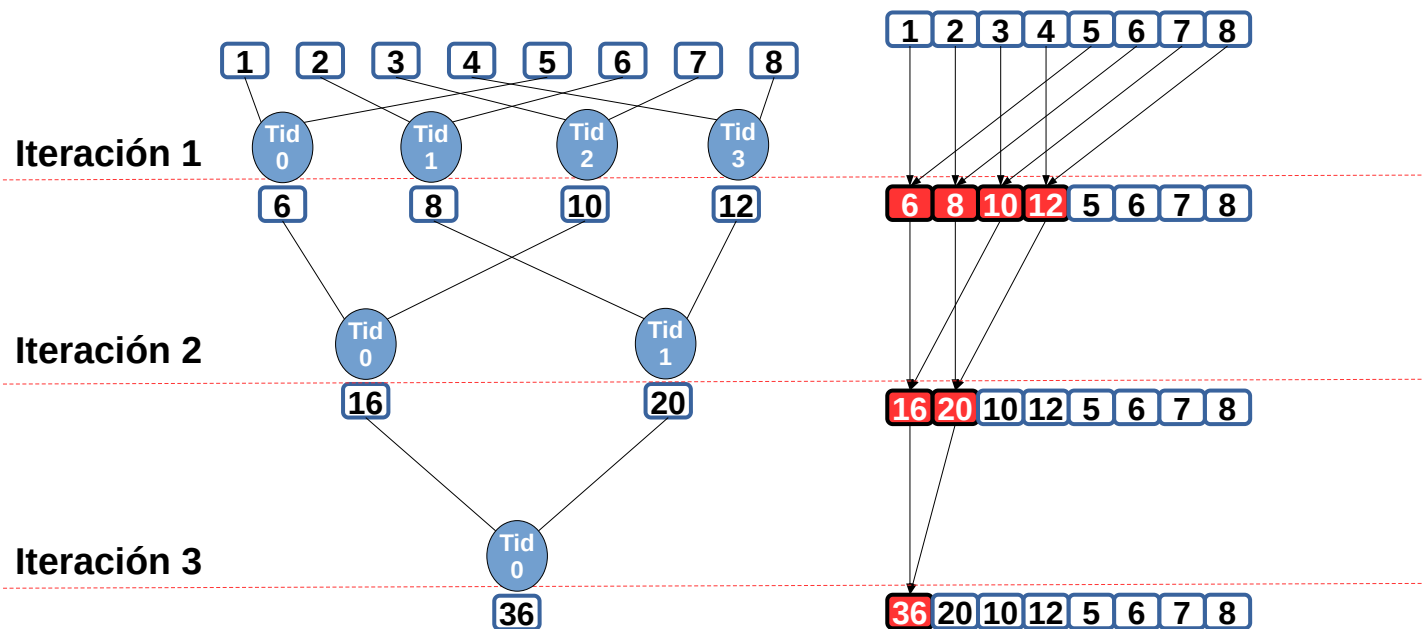
- Las siguientes iteraciones reducen los resultados de los bloques de la iteración anterior.



Suma por reducción

Mejoras por el uso de memoria compartida

- Usamos el siguiente patrón de acceso que asegura un direccionamiento secuencial libre de conflictos.



Suma por reducción

Implementación

```
__global__ void reduce(int *g_idata, int *g_odata) {  
    extern __shared__ int sdata[];  
    unsigned int tid = blockIdx.x*blockDim.x + threadIdx.x;
```

```
    sdata[threadIdx.x] = g_idata[tid];  
    __syncthreads();
```

```
    for(unsigned int s=blockDim.x/2; s >0; s >>= 1) {
```

```
        if (threadIdx.x < s)
```

```
            sdata[threadIdx.x] += sdata[threadIdx.x + s];
```

```
        __syncthreads();
```

```
    }
```

```
    if (threadIdx.x == 0)
```

```
        g_odata[blockIdx.x] = sdata[0];
```

```
}
```

if con bajo grado de divergencia

